

EXERCICE 1 :

- 1) Rappeler les bornes minimale et maximale pour le codage des nombres signés et non signés sur n bits.
- Pour un nombre **signé** sur n bits, les valeurs vont de $-2^{(n-1)}$ à $2^{(n-1)}-1$.
 - Pour un nombre **non signé** sur n bits, les valeurs vont de 0 à 2^n-1 .
- 2) et 3) Conversion des nombres en base 10 sur 10 bits et hexadécimale:
- Avec 10 bits, on peut coder des nombres **signés** allant de **-512** à **511**.
 - Avec 10 bits, on peut coder des nombres **non signés** allant de **0** à **1023**.

	Signé	Non signé	hexadécimale
344	Oui, 0101011000	Oui, 0101011000	158
115	Oui, 0001110011	Oui, 0001110011	73
-42	Oui, 1111010110	Non	FD6
666	Non (hors de la plage)	Oui, 1010011010	29A
-950	Non (hors de la plage)	Non	x
-496	Oui, 1000010000	Non	E10
260	Oui, 0100000100	Oui, 0100000100	104
1515	Non (hors de la plage)	Non	x

- 4) Pour chaque nombre suivant en base 2 sur 8 bits signés, donner sa valeur numérique correspondante en base 10 :

- On utilise la méthode du complément à 2

0b11000011 = - **61**, 0b10100110 = -**90**, 0b11110000 = -**16**, 0b11111101 = -**3**, 0b00111011 = **59**

- 5) En VHDL et en Verilog, comment convertit-on les nombres décimaux 293993 et -293900 en nombres binaires signés et non signés ?

- **VHDL**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Convertisseur is
    Port (
        resultat_non_signe : out std_logic_vector(18 downto 0);
        resultat_signe : out std_logic_vector(19 downto 0)
    );
end Convertisseur;

architecture Behavioral of Convertisseur is
    signal decimal_value : integer := 293993;
    signal binary_value_unsigned : std_logic_vector(18 downto 0);
    signal binary_value_signed : std_logic_vector(19 downto 0);
begin
```

```

binary_value_unsigned <= std_logic_vector(to_unsigned(decimal_value, 19));
binary_value_signed <= std_logic_vector(to_signed(decimal_value, 20));
resultat_non_signe <= binary_value_unsigned;
resultat_signe <= binary_value_signed;
end Behavioral;
    
```

• Verilog

```

module Convertisseur (
    output reg [18:0] resultat_non_signe,
    output reg [19:0] resultat_signe
);
integer decimal_value = 293993;
always @(*) begin
    resultat_non_signe = decimal_value[18:0];
    resultat_signe = decimal_value[19:0];
end
endmodule
    
```

EXERCICE 2 :

1) Rappeler la table de vérité de la fonction NAND.

a	b	NAND
0	0	1
0	1	1
1	0	1
1	1	0

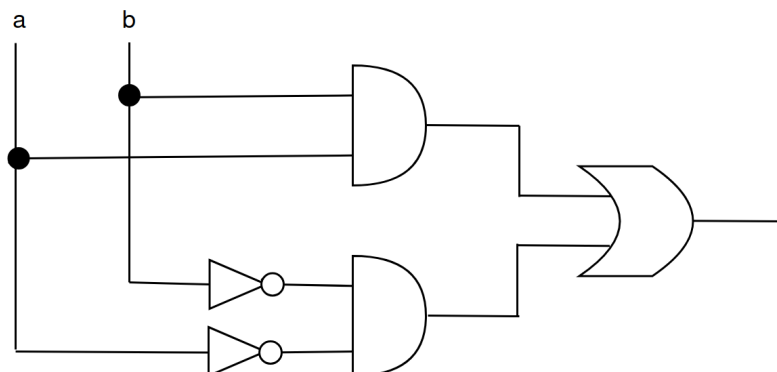
2) Démontrer par les tables de vérité l'égalité suivante : $a \oplus b = a \cdot b + a \cdot b'$

a	b	a'	b'	a xor b	a' · b	a · b'	a' · b + a · b'
0	0	1	1	0	0	0	0
0	1	1	0	1	1	0	1
1	0	0	1	1	0	1	1
1	1	0	0	0	0	0	0

3) En déduire l'expression et un circuit à base de AND, OR et NOT de la fonction XNOR

- En utilisant le tableau de Karnaugh basé sur la table de vérité de l'opération XNOR : $(a \cdot b) + (\bar{a} \cdot \bar{b})$.
- Remarque : Il est également possible d'appliquer la loi de Morgan à l'équation logique de l'opération XOR. : $(a + \bar{b}) \cdot (\bar{a} + b)$

Pour le circuit, on peut réaliser l'équation dérivée du tableau de Karnaugh.



4) Donner les tables de vérités des fonctions suivantes : $a + a \cdot b$, $a + \bar{a} \cdot b$, multiplexeur 4 vers 1.

a	b	$a + a \cdot b$
0	0	0
0	1	0
1	0	1
1	1	1

a	b	$\bar{a}$	$\bar{a} \cdot b$	$a + \bar{a} \cdot b$
0	0	1	0	0
0	1	1	1	1
1	0	0	0	1
1	1	0	0	1

Sel0	Sel1	Sortie
0	0	In0
0	1	In1
1	0	In2
1	1	In3

EXERCICE 3 :

On souhaite réaliser un comparateur d'égalité de 2 valeurs sur 4 bits qui renvoie 1 si les 2 valeurs sont égales et 0 sinon.

1) Quelle fonction logique booléenne retourne 0 lorsque les entrées sont différentes et 1 sinon.

- La fonction est XNOR.

2) Combien de lignes comporte la table de vérité complète ?

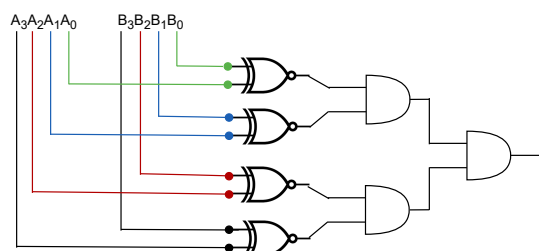
- Considérons les valeurs suivantes qui sont sur 4 bits : $A = A_3A_2A_1A_0$ et $B = B_3B_2B_1B_0$, la taille de la table de vérité aura donc $2^{4+4} = 256$ lignes.

3) Donner la fonction logique correspondante à la comparaison sur 4 bits.

- Pour que les deux valeurs soient similaires, il faut que chaque bit à l'indice n de A soit égal au bit à l'indice n de B .
- La fonction logique est :

$$\text{Comparateur} = (A_3 \text{ XNOR } B_3) \text{ AND } (A_2 \text{ XNOR } B_2) \text{ AND } (A_1 \text{ XNOR } B_1) \text{ AND } (A_0 \text{ XNOR } B_0).$$

4) Faire le schéma de câblage



5) Ecrire cette fonction en VHDL et Verilog comportemental.

- **VHDL :**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Compareteur_4bits is
    Port (
        A : in STD_LOGIC_VECTOR(3 downto 0);
        B : in STD_LOGIC_VECTOR(3 downto 0);
        Equal : out STD_LOGIC
    );
end Compareteur_4bits;

architecture Behavioral of Compareteur_4bits is
begin
    Equal <= (A(3) xnor B(3)) and (A(2) xnor B(2)) and (A(1) xnor B(1)) and (A(0) xnor B(0));
end Behavioral;
```

- **Verilog :**

```
module Compareteur_4bits (
    input [3:0] A,
    input [3:0] B,
    output Equal
);

    assign Equal = (A[3] ~^ B[3]) & (A[2] ~^ B[2]) & (A[1] ~^ B[1]) & (A[0] ~^ B[0]);
endmodule
```

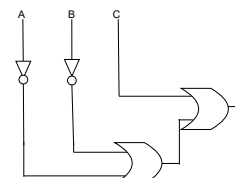
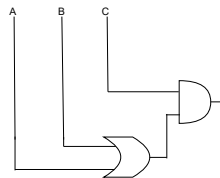
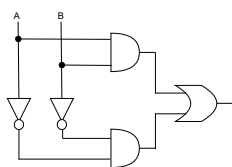
EXERCICE 4 :

$$\text{Eq1 (a,b)} = a \cdot b + \bar{a} \cdot \bar{b}$$

$$\text{Eq2 (a,b,c)} = (a+b) \cdot c$$

$$\text{Eq3 (a,b,c)} = \overline{a \cdot b} + c$$

1) Représentez avec des portes logiques les trois équations.



2) Codez en VHDL et Verilog les équations en comportemental .

- On le fait pour l'équation 1
- **Vhdl**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Eq1 is
```

```

Port (
    a : in STD_LOGIC;
    b : in STD_LOGIC;
    result : out STD_LOGIC
);

end Eq1;

architecture Behavioral of Eq1 is
begin
    result <= (a AND b) OR (NOT a AND NOT b);
end Behavioral;
    
```

- **Verilog**

```

module Eq1 (
    input wire a,
    input wire b,
    output wire result
);
    assign result = (a & b) | (~a & ~b);
endmodule
    
```

EXERCICE 5 :

Un afficheur 7 segments est un dispositif permettant d'afficher les nombres décimaux ou hexadécimaux avec une entrée sur 4 bits (X3, X2, X1, X0) et une sortie sur 7 bits (a, b, c, d, e, f, g) :

1) Compléter la table de vérité ci-dessus.

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	0	1	0	1	1	0	1	1	0	1	2
3	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	0	1	1	1	1	1	6
7	0	1	1	1	1	1	1	0	0	0	0	7
8	1	0	0	0	1	1	1	1	1	1	1	8
9	1	0	0	1	1	1	1	1	0	1	1	9

2) En utilisant les tableaux de Karnaugh, simplifier les fonctions des segments a, b et g.

